Python Automation, Scheduling, Logging, and Web Scraping

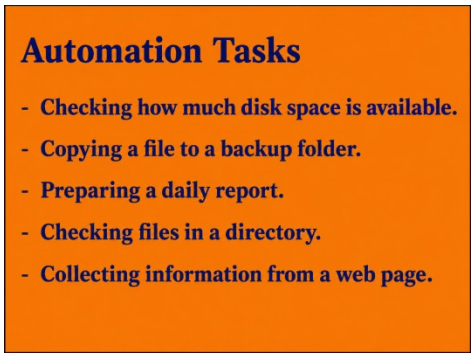
What is Automation?

Automation means writing instructions that perform a task with less manual effort. In DevOps, repeated tasks such as checking disk space, backing up files, preparing daily reports and collecting data from web pages are automated to save effort and avoid mistakes. **Python** performs the steps, a **scheduler** decides when the script runs, and **logs** record what happened.

Before automating a task, define:

- **Inputs:** what information the script receives.
- **Sources and Outputs:** which files or folders it uses, and where results are saved.
- **Error Handling:** what happens if something is missing.
- **Idempotency:** what happens when the script runs repeatedly.

Workflow: Read Input → Check Input → Perform Task → Save Result → Record Log



Practice Setup and Paths (pathlib)

Work in a separate folder with an isolated virtual environment (Python 3 on Ubuntu). The `pathlib` module handles file paths as objects, and the `/` operator joins path parts safely.

```
mkdir week4 && cd week4
python3 -m venv .venv
source .venv/bin/activate
```

Method	Purpose
`exists()`	Checks whether the path exists.
`is_file()` / `is_dir()`	Checks for a regular file / a directory.
`mkdir()`	Creates a directory (`parents=True, exist_ok=True`).
`iterdir()` / `glob()`	Lists directory contents / matches a pattern such as `*.txt`.
`stat()`	Returns file metadata such as `st_size`.

Create a separate folder and set up an isolated virtual environment:

1. Create a new folder and navigate into it:

```
$ mkdir week4
$ cd week4
```

2. Create and activate virtual environment:

```
$ python3 -m venv .venv
$ source .venv/bin/activate
```

Files, Backups and shutil

Scripts can find files with `glob("*.txt")` and read sizes with `stat().st_size`. The `shutil` module handles high-level file operations:

- `copyfile()`: copies content only; `copy()`: content and permissions.
- `copy2()`: content, permissions and modification timestamps.
- `move()`: moves a file or directory; `disk_usage()`: total, used and free space.

Backup example: confirm the source is a file, create the backup folder, refuse to overwrite an existing backup, then call `shutil.copy2()`.

- **Dry Run:** prints planned actions without changing any files.
- **Idempotent Operation:** gives the same result however many times it runs (e.g. `mkdir(exist_ok=True)`).

Workflow: Validate Source → Create Folder → Check Existing Backup → Copy

- **copyfile(src, dst):**
  Copies file content only.
- **copy(src, dst):**
  Copies content and permission flags.
- **copy2(src, dst):**
  Copies content, permissions, and preserves modification timestamps.
- **move(src, dst):**
  Recursively moves a file or directory.
- **disk_usage(path):**
  Returns total, used, and free space on the filesystem.

```
from pathlib import Path
import shutil

source = Path("input/alpha.txt")
destination = Path("backup/alpha.txt")

if not source.is_file():
    raise FileNotFoundError("The source file is missing")

# Ensure destination directory exists
destination.parent.mkdir(parents=True, exist_ok=True)

if destination.exists():
    raise FileExistsError("The backup already exists")

shutil.copy2(source, destination)
print("Backup created:", destination)
```

System Checks and Running Commands

A disk check reads `shutil.disk_usage(".")`, calculates the used percentage and prints a **warning at 80% or above**. To run external programs, use `subprocess.run()` with the command as a list so no raw shell is opened.

- Use `check=True` to raise an error if the command fails.
- Use `capture_output=True, text=True` to read the output as text.
- Use `timeout` and catch `CalledProcessError` and `TimeoutExpired`.

```
subprocess.run(["git", "--version"], check=True,
               capture_output=True, text=True, timeout=5)
```

```
import subprocess

try:
    result = subprocess.run(
        ["git", "--version"],
        check=True,
        capture_output=True,
        text=True,
        timeout=5,
    )
    print("Git version:", result.stdout.strip())

except subprocess.CalledProcessError as error:
    print("Command failed with return code:", error.returncode)

except subprocess.TimeoutExpired:
    print("Command timed out")
```

Logging

A **report** presents the result of a task (e.g. `Files: 2 | Total bytes: 13`), while a **log** records runtime steps, timing and errors over time. Logging levels show how serious a message is:

Level	Value	Purpose
DEBUG	10	Detailed diagnostic information for troubleshooting.
INFO	20	Normal progress and milestone messages.
WARNING	30	Unexpected event that does not stop execution.
ERROR	40	A task failed due to an exception.
CRITICAL	50	Severe failure threatening the application.

To stop log files growing endlessly, use **RotatingFileHandler** (rotates at a size limit) or **TimedRotatingFileHandler** (rotates at intervals, e.g. midnight).



```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Automation script initiated")

**14. Worked Example: File Inventory with Logging**
Save this script as inventory.py:

import json
import logging
from pathlib import Path
import sys

base = Path(__file__).resolve().parent

logging.basicConfig(
    filename=base / "automation.log",
    encoding="utf-8",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)

def main():
    source = base / "input"
    try:
        if not source.is_dir():
            raise FileNotFoundError("The input folder is missing")
        logging.info("Inventory started")
        files = []
        for path in sorted(source.iterdir()):
            if (
                path.suffix == ".txt"
                and path.is_file()
                and not path.is_symlink()
            ):
                files.append({
                    "name": path.name,
                    "bytes": path.stat().st_size,
                })
        report = {
            "file_count": len(files),
            "files": files,
            "total_bytes": sum(item["bytes"] for item in files),
            }
        with open(base / "inventory.json", "w", encoding="utf-8") as output:
            json.dump(report, output, indent=2)
        logging.info("Inventory finished: %s files processed", len(files))
        print(f"Files: {len(files)}; bytes: {report['total_bytes']}")
        return 0
    except OSError:
        logging.exception("Inventory failed due to an OS error")
        print("Inventory failed; inspect automation.log", file=sys.stderr)
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

Worked Example: File Inventory with Logging

`inventory.py` scans the `input` folder for `.txt` files, writes a JSON report (`file_count`, `files`, `total_bytes`) and records its activity in `automation.log`.

- `Path(__file__).resolve().parent` keeps paths correct from any working directory.
- `is_symlink()` filtering skips broken shortcuts.
- `logging.exception()` adds the full error traceback to the log.
- The script returns `0` on success and `1` on failure for shell runners.

Workflow: Check Folder → Scan .txt Files → Write inventory.json → Write automation.log

```
import json
import logging
from pathlib import Path
import sys

base = Path(__file__).resolve().parent

logging.basicConfig(
    filename=base / "automation.log",
    encoding="utf-8",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)


def main():
    source = base / "input"

    try:
        if not source.is_dir():
            raise FileNotFoundError("The input folder is missing")

        logging.info("Inventory started")
        files = []

        for path in sorted(source.iterdir()):
            if (
                path.suffix == ".txt"
                and path.is_file()
                and not path.is_symlink()
            ):
                files.append({
                    "name": path.name,
                    "bytes": path.stat().st_size,
                })

        report = {
            "file_count": len(files),
            "files": files,
            "total_bytes": sum(item["bytes"] for item in files),
        }

        with open(base / "inventory.json", "w", encoding="utf-8") as output:
            json.dump(report, output, indent=2)

        logging.info("Inventory finished: %s files processed", len(files))
        print(f"Files: {len(files)}; bytes: {report['total_bytes']}")
        return 0

    except OSError:
        logging.exception("Inventory failed due to an OS error")
        print("Inventory failed; inspect automation.log", file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

Scheduling with Cron

Cron is a daemon that runs scheduled commands on Unix-like systems. Each crontab line has five time fields followed by the command:

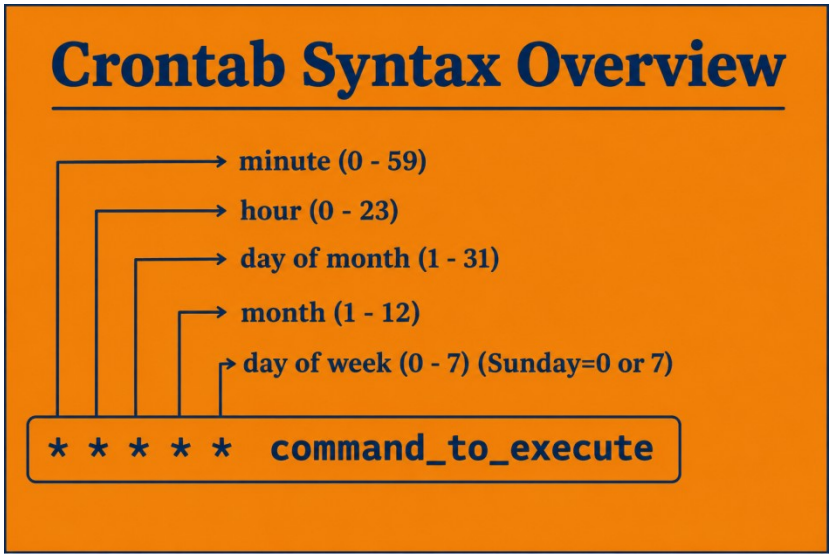
```
* * * * *  command_to_execute
| | | | |
| | | | +-- day of week (0-7, Sunday = 0 or 7)
| | | +---- month (1-12)
| | +------ day of month (1-31)
| +-------- hour (0-23)
+---------- minute (0-59)
```

- `*/15 * * * *` : every 15 minutes.
- `30 8 * * 1-5` : 08:30 AM, Monday to Friday.
- `0 9,18 * * *` : twice daily at 09:00 AM and 06:00 PM.

Running Scripts with Cron

Edit the user crontab with `crontab -e`. **Key rule:** call the virtual environment's interpreter by absolute path (`.venv/bin/python`) and redirect `>> cron-output.log 2>&1` to capture errors. Common failures:

1. **Environment mismatch:** cron does not load `.bashrc` or shell profiles.
2. **Relative paths:** the default working directory breaks relative path logic.
3. **Missing binaries:** `PATH` is minimal, so use full paths.
4. **Overlapping runs:** a long task may still run when the next one starts.



Web Scraping with Beautiful Soup

Web scraping extracts information from HTML. **Beautiful Soup** (`bs4`, installed with `pip install beautifulsoup4`) parses static HTML; it does **not** run JavaScript. Use `soup.select("#students .student")` to pick elements and `get_text(strip=True)` to read them, and raise an error if no element is found.

- Check `robots.txt` and respect rate limits.
- Set connection and read timeouts on requests.
- For JavaScript-driven pages, use browser automation.

Workflow: Fetch Page → Check Status (200) → Parse HTML → Validate Selectors → Export Data

BeautifulSoup: Extract Student Names

PYTHON • HTML PARSING

```
from bs4 import BeautifulSoup

html_doc = """
<ul id="students">
  <li class="student">Riya</li>
  <li class="student">Aman</li>
</ul>
"""

soup = BeautifulSoup(html_doc, "html.parser")
students = soup.select("#students .student")

if not students:
    raise ValueError("Expected at least one student element in markup")

for student in students:
    print(student.get_text(strip=True))
```

Common Mistakes and Quick Revision

Common mistakes to avoid:

1. Relying on the working directory instead of `Path(__file__)`.
2. Opening files in `"w"` mode before validating input.
3. Hiding errors with `except: pass`.
4. Using relative interpreter paths in cron jobs.
5. Letting log files grow without rotation.